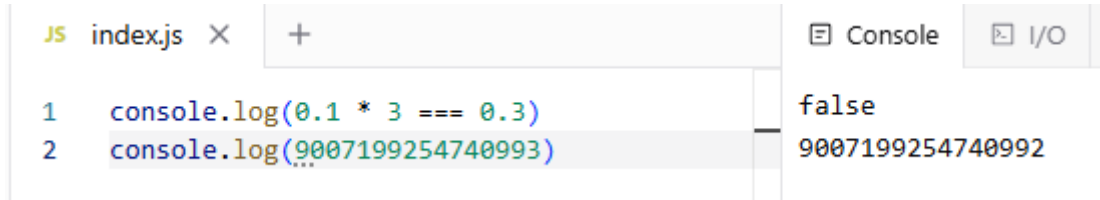


Rafael Blasques Morales

RA: 24124872-2

1 – a) Saída:



```
JS index.js x +
1 console.log(0.1 * 3 === 0.3)
2 console.log(9007199254740993)
```

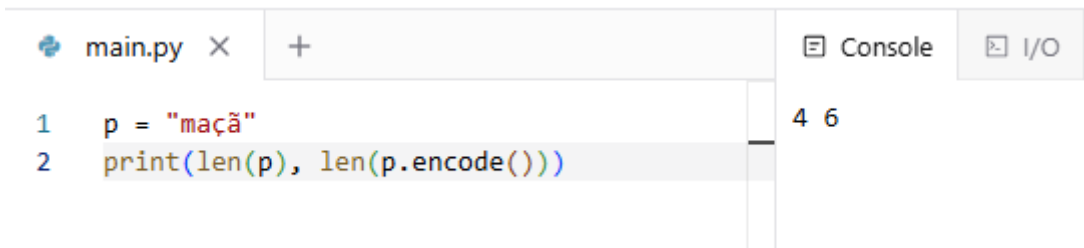
Console I/O

false
9007199254740992

b) O problema ocorre na execução. No primeiro console log, por ser uma dízima periódica, o computador não consegue lidar com o último dígito, então o resultado guarda um valor mínimo final.

No segundo console log, o valor ultrapassa o valor máximo base do console log

2 – a) Saída:



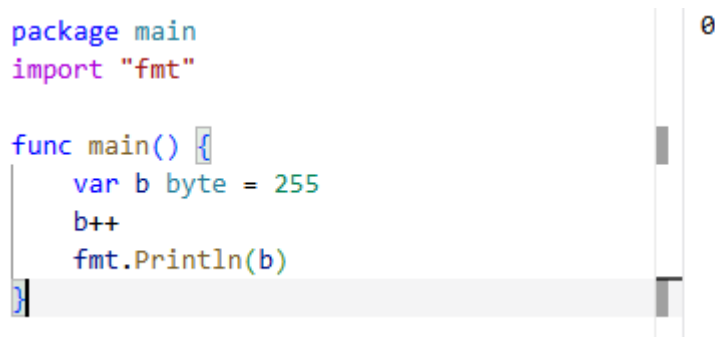
```
main.py x +
1 p = "maçã"
2 print(len(p), len(p.encode()))
```

Console I/O

4 6

b) Não existe problema

3 – a) Saída:



```
package main
import "fmt"

func main() {
    var b byte = 255
    b++
    fmt.Println(b)
}
```

0

b) O problema ocorre em tempo de execução. O tipo byte em GO, é 8 bits, aceitando somente valores de 0 a 255. Quando b++ executa, o valor volta ao início do limite

4 – a) Saída:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] v = new int[3];
        System.out.println(v[0]);
        System.out.println(v[3]);
    }
}
```

0
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException: Index 3
out of bounds for length 3
at Main.main(Main.java:7)
Debug this error

b) O erro ocorre na compilação. Declarar `int[3]`, cria o array com 3 espaços, de `[0,1,2]` basicamente, então o `v[3]` não existe. Se quiser acessar a terceira posição do array, usa o `v[2]`

5 – a) Saída:

```
HelloWorld.rs x Console I/O AI Agent Exit 1

1 fn main() {
2     let s = String::from("oi");
3     let t = s;
4     println!("{}", s, t);
5 }
```

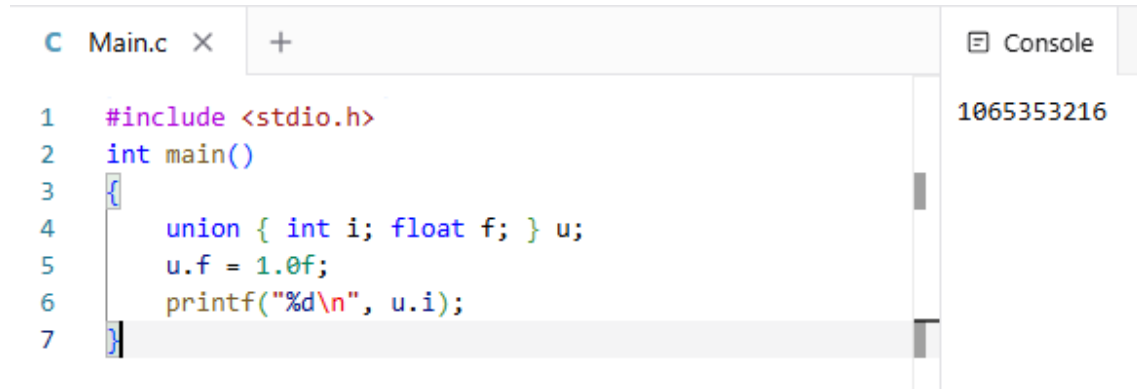
error[E0382]: borrow of moved value: `s`
--> HelloWorld.rs:4:23
|
2 | let s = String::from("oi");
| - move occurs because `s` has type `String`,
| which does not implement the `Copy` trait
3 | let t = s;
| - value moved here
4 | println!("{}", s, t);
| ^ value borrowed here after
| move
|
help: consider cloning the value if the performance cost
| is acceptable
3 | let t = s.clone();
| +++++++

error: aborting due to 1 previous error

For more information about this error, try `rustc --
explain E0382`.
Debug this error

b) O erro ocorre na compilação. Isso ocorre por conta da transferência de posse de uma string, quando fazemos `t = s`, o conteúdo de “s” vai para “t”. Para copiar a string, poderíamos utilizar alguma função própria

6 – a) Saída:



```
C Main.c x + Console
1  #include <stdio.h>
2  int main()
3  {
4      union { int i; float f; } u;
5      u.f = 1.0f;
6      printf("%d\n", u.i);
7  }
```

1065353216

b) O erro ocorre na execução do programa. Isso acontece porque `u.f` e `u.i` compartilham a mesma memória. Ao armazenar `1.0f` em `u.f` e depois acessar `u.i`, os mesmos bits são interpretados como `int`, o que pode gerar um valor inesperado.